Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram

(Multimedia Analysis – Project Report)

Multi-Modal Human Pose & Gesture Estimation System

Real-Time Analysis with Web Dashboard, Sign Language Recognition & Gesture-Controlled Game

---

NAME

Roll Number

March 16, 2026

Declaration

I hereby declare that this project report titled **"Multi-Modal Human Pose & Gesture Estimation System"** is a bonafide record of work carried out by me as part of the course **Multimedia Analysis** at **IIITDM Kancheepuram**. This report has not been submitted elsewhere for the award of any other degree or diploma.

Acknowledgment

I would like to express my sincere gratitude to the course instructor for continuous support and encouragement throughout the course of this project.

I am also thankful to the **Department of Computer Science and Engineering, IIITDM Kancheepuram**, for providing the opportunity and facilities to undertake this project.

Special thanks to the open-source communities behind MediaPipe, OpenCV, and Flask for providing the foundational tools that made this system possible.

NAME

Roll Number

Abstract

This report presents a comprehensive **multi-modal real-time human analysis system** that simultaneously processes body pose, hand gestures, facial expressions, sign language recognition, and gesture-controlled interaction from a single webcam feed. The system employs Google's **MediaPipe** framework – specifically BlazePose, Hand Tracking, and FaceMesh models – to extract skeletal, hand, and facial landmarks, which are then analyzed through a pipeline of custom modules.

The analysis pipeline computes **9 joint angles** with EMA smoothing, classifies **12+ hand gestures**, detects facial expressions via **Eye Aspect Ratio (EAR)** and **Mouth Aspect Ratio (MAR)**, counts exercise repetitions using angle-based state machines, analyzes bilateral pose symmetry, and recognizes **20 ASL alphabet letters** through geometric hand landmark classification.

A **Flask-based web dashboard** provides an interactive browser interface with live MJPEG video streaming, analytics panels, a sign language translator, and a gesture-controlled dodging game. The system comprises **25+ source files**, supports both standalone OpenCV and web-based modes, and delivers real-time performance at 15–25 FPS on standard hardware.

Keywords: Pose Estimation, MediaPipe, Gesture Recognition, Sign Language, EAR, MAR, Flask, Computer Vision, Real-Time Analysis, Exercise Tracking.

Table of Contents

Abstract **2**

1 Introduction **1**

1.1 Project Overview 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 System Features at a Glance 2

2 Literature Review and Background **4**

2.1 MediaPipe Framework 4

2.2 BlazePose 4

2.3 Hand Tracking 4

2.4 Face Mesh 4

2.5 Eye Aspect Ratio 4

2.6 Related Work 5

3 System Design and Architecture **6**

3.1 Overall Architecture 6

3.2 Project Structure 6

3.3 Data Flow Pipeline 7

3.4 Web Dashboard Architecture 7

4 Mathematical Foundations **8**

4.1 Joint Angle Computation (2D Kinematics) 8

4.2 3D Joint Angle (Vector Algebra) 8

4.3 Eye Aspect Ratio (EAR) 8

4.4 Mouth Aspect Ratio (MAR) 8

4.5 Exponential Moving Average (EMA) 9

4.6 Head Tilt (Roll Angle) 9

4.7 Exercise Rep Detection State Machine 9

4.8 Bilateral Symmetry Score 9

4.9 Finger Curl Ratio 9

4.10 Temporal Debounce 10

5 Implementation Details **11**

5.1 Pose Estimation Module (`pose.py`) 11

5.2 Hand Gesture Recognition (`hands.py`) 11

5.3 Face Expression Detection (face.py) 12
5.4 Activity Classification (activity.py) 12
5.5 Exercise Repetition Counter (exercise.py) 13
5.6 Symmetry Analysis (symmetry.py) 13
5.7 Sign Language Recognition (sign_language.py) 13

6 Web Dashboard **14**
6.1 Technology Stack 14
6.2 Dashboard Pages 14
6.2.1 Home Page 14
6.2.2 Camera Analysis Page 14
6.2.3 Sign Language Page 15
6.2.4 Gesture Game Page 15
6.3 API Endpoints 16

7 Gesture-Controlled Game **17**
7.1 Game Design 17
7.2 Body-to-Game Mapping 17
7.3 Game Mechanics 17

8 Advanced Visualizations **18**
8.1 Motion Trajectory Trails 18
8.2 Real-Time Scrolling Graphs 18
8.3 Multi-Panel HUD 18

9 Testing and Results **19**
9.1 Test Methodology 19
9.2 Automated Test Results 19
9.3 Performance Metrics 20

10 Challenges and Solutions **21**
10.1 Technical Challenges 21
10.1.1 Landmark Jitter 21
10.1.2 Activity Classification Flickering 21
10.1.3 Thread-Safe Video Streaming 21
10.1.4 Sign Language Ambiguity 21
10.2 Design Challenges 21
10.2.1 Modular Integration 21
10.2.2 Web vs Standalone Mode 22

11 Conclusion and Future Scope **23**

11.1 Project Summary 23
11.2 Technical Contributions 23
11.3 Lessons Learned 23
11.4 Future Work 24

A Installation and Setup Guide **25**
A.1 System Requirements 25
A.2 Installation Steps 25
A.3 Command Reference 26

B Key Code Listings **27**
B.1 EMA Filter Implementation 27
B.2 Exercise Rep Counter State Machine 27
B.3 Flask MJPEG Stream Generator 27

C Premium HCI & Form-Checking Heuristics **29**
C.1 Posture and Spine Alignment Heuristic 29
C.2 Landmark-Based Squat Depth Validation 29
C.3 Phase 6: Advanced ASL Expansion (20 Letters) 29
C.4 Premium Visual Aesthetics: Neon Skeleton Drawing 30

References **31**

List of Figures

1.1 Overview of the Real-Time Analysis Pipeline Flow 3

6.1 Web Dashboard Home Page – Feature cards and system statistics 14
6.2 Camera Analysis Page – Live video with analytics panels 15
6.3 Sign Language Recognition Page – Letter detection and sentence builder 15
6.4 Gesture-Controlled Game Page – Canvas game and body control indicators 16

List of Tables

1.1 System Feature Summary 2

2.1 Comparison with Existing Pose Estimation Systems 5

5.1 Gesture Classification Rules 12
5.2 Activity Classification Heuristics 12
5.3 Exercise Configurations (Equation 4.7) 13

6.1 Flask API Endpoint Reference 16

9.1 Automated Test Results 19
9.2 System Performance Metrics 20

A.1 Command and Keyboard Shortcut Reference 26

1. Introduction

1.1 Project Overview

The **Multi-Modal Human Pose & Gesture Estimation System** is an advanced real-time computer vision application that integrates multiple analysis engines to provide comprehensive human body understanding from a single webcam feed. Unlike traditional single-purpose pose estimators, this system simultaneously performs body pose estimation, hand gesture recognition, facial expression detection, activity classification, exercise tracking, symmetry analysis, sign language recognition, and gesture-controlled gaming.

The project is built on Google's MediaPipe framework and extends its capabilities through a modular pipeline of custom analysis engines, each implementing domain-specific algorithms and heuristics.

1.2 Problem Statement

Human pose estimation and gesture recognition are fundamental problems in computer vision with applications in:

- **Healthcare:** Physical therapy monitoring, ergonomic assessment
- **Accessibility:** Sign language translation for the hearing impaired
- **Fitness:** Automated exercise form checking and repetition counting
- **Entertainment:** Gesture-controlled gaming and interactive experiences
- **HCI:** Natural human-computer interaction without controllers

Traditional approaches require specialized hardware (depth cameras, motion capture suits) or separate systems for each modality. This project addresses the challenge of integrating all these capabilities into a single, real-time, webcam-based system.

1.3 Objectives

The primary objectives of this project are:

1. **Multi-Joint Kinematic Analysis:** Compute 9 joint angles with real-time smoothing
2. **Gesture Vocabulary:** Classify 12+ hand gestures from landmark geometry

3. **Expression Detection:** Detect blinks, yawns, talking, and head tilt using EAR/-MAR

4. **Activity Classification:** Identify 7 body postures with temporal debouncing

5. **Exercise Counting:** Track repetitions for 5 exercises using state machines

6. **Symmetry Analysis:** Compute bilateral joint symmetry scores

7. **Sign Language:** Recognize 20 ASL alphabet letters geometrically

8. **Gesture Game:** Create body-controlled interactive game

9. **Web Dashboard:** Build browser-based interface with live analytics

10. **Visualization:** Implement motion trails and real-time scrolling graphs

1.4 System Features at a Glance

Feature	**Value**
Total Source Files	25+
Core Analysis Engines	7
Utility Modules	6
Web Dashboard Pages	4
Joint Angles Tracked	9
Gesture Vocabulary	12+
Activity Classifications	7
ASL Letters Supported	20
Exercise Types	5
Symmetry Joint Pairs	4
Keyboard Shortcuts (Standalone)	11
Recording Modes	3 (CSV, Video, Snapshot)
Real-Time Graphs	4
Face Landmarks	468

Table 1.1: System Feature Summary

ANALYSIS PIPELINE FLOW

`Webcam` → `Frame Capture` → `MediaPipe Models` → `Custom Analysis Engines` → `HUD / Web Dashboard`

Pose (33 lm) — Hands (21 lm) — Face (468 lm) → Angles, Gestures, EAR/MAR, Activity, Exercise, Symmetry, Sign Language

Figure 1.1: Overview of the Real-Time Analysis Pipeline Flow

2. Literature Review and Background

Human pose estimation has evolved from classical approaches using Histogram of Oriented Gradients (HOG) and deformable part models to modern deep learning architectures. Recent advances in lightweight neural networks have made real-time inference possible on consumer hardware.

2.1 MediaPipe Framework

Google's MediaPipe [5] provides a modular framework for building perception pipelines. It offers pre-trained models for face detection, hand tracking, and body pose estimation that run efficiently on CPUs and mobile GPUs.

2.2 BlazePose

BlazePose [1] detects 33 body landmarks in real-time using a lightweight CNN architecture. It achieves comparable accuracy to heavier models while maintaining interactive frame rates, making it ideal for real-time applications.

2.3 Hand Tracking

MediaPipe Hands [7] provides 21-landmark hand tracking using a two-stage pipeline: palm detection followed by precise landmark regression. This enables detailed finger state analysis and gesture classification.

2.4 Face Mesh

The FaceMesh model [4] detects 468 facial landmarks in real-time, enabling detailed facial geometry analysis including eye tracking, mouth state, and head orientation estimation.

2.5 Eye Aspect Ratio

Soukupová and Čech [6] introduced the Eye Aspect Ratio (EAR) formula for blink detection, using the ratio of vertical to horizontal eye distances. This metric provides a reliable, threshold-based approach to drowsiness and attentiveness detection.

2.6 Related Work

Feature	**Our System**	**OpenPose**	**PoseNet**	**MMPose**
Real-time	Yes	Yes	Yes	Partial
Hand Gestures	12+ classes	No	No	Limited
Face Analysis	EAR/MAR	No	No	Limited
Sign Language	20 letters	No	No	No
Exercise Tracking	Yes	No	No	No
Web Dashboard	Yes	No	No	No
Game Control	Yes	No	No	No
Symmetry Analysis	Yes	No	No	No

Table 2.1: Comparison with Existing Pose Estimation Systems

3. System Design and Architecture

3.1 Overall Architecture

The system follows a modular architecture with clear separation of concerns. Each analysis engine is self-contained and communicates through standardized data interfaces.

3.2 Project Structure

```
MM/
  app.py                    # Flask web dashboard entry point
  main.py                   # Standalone OpenCV mode
  requirements.txt          # Dependencies
  src/
    core/
      pose.py               # PoseTracker (BlazePose + kinematics + seg
          )
      hands.py              # GestureTracker (12+ gestures)
      face.py               # FaceTracker (EAR/MAR/expression)
      activity.py           # ActivityClassifier (7 postures)
      exercise.py           # ExerciseCounter (5 exercises)
      symmetry.py           # SymmetryAnalyzer (bilateral)
      sign_language.py       # SignLanguageClassifier (20 ASL letters)
    utils/
      angles.py             # 2D/3D angle computation
      smoothing.py          # EMA filter bank
      recorder.py           # CSV + MP4 video recorder
      visuals.py            # Multi-panel HUD renderer
      trails.py             # Trajectory trail renderer
      graphs.py             # Real-time scrolling graphs
    web/
      routes.py             # Flask routes + JSON API
      stream.py             # Background video stream manager
  templates/                # 4 HTML dashboard pages
  static/
    css/style.css           # Premium dark theme
    js/camera.js            # Camera analytics logic
    js/sign_language.js     # Sign language UI
    js/game.js              # Canvas dodging game
  report/                   # LaTeX report
  screenshots/              # Dashboard screenshots
```

Listing 3.1: Complete Project File Structure

3.3 Data Flow Pipeline

Algorithm 1 Main Analysis Loop (per frame)

```
Capture frame from webcam, mirror horizontally
if Pose enabled: Extract 33 body landmarks via BlazePose
Compute 9 joint angles using 2D kinematics (Eq. 4.1)
Apply EMA smoothing to all angle values (Eq. 4.5)
Classify activity using landmark geometry + debounce
Update exercise rep counters via state machines
Compute bilateral symmetry scores
Apply background segmentation (blur/gradient) if enabled
if Hands enabled: Extract 21 landmarks per hand
Classify gesture from finger states and distances
Run ASL letter classification on hand geometry
if Face enabled: Extract 468 face landmarks
Compute EAR (Eq. 4.3), MAR (Eq. 4.4), head tilt
Classify expression (Neutral/Blink/Yawn/Talk/Tilt)
Render trails, graphs, HUD overlays onto frame
Output frame to display / MJPEG stream / recorder
```

3.4 Web Dashboard Architecture

The web dashboard uses Flask with MJPEG streaming for the video feed and JSON API endpoints for analytics data:

- **Backend:** Flask 3.x with Blueprint-based routing
- **Video Stream:** MJPEG over HTTP (`multipart/x-mixed-replace`)
- **Analytics API:** JSON endpoints polled every 250ms by frontend JavaScript
- **Background Thread:** Camera capture and analysis run in a daemon thread
- **Thread Safety:** Shared state protected by `threading.Lock`

4. Mathematical Foundations

4.1 Joint Angle Computation (2D Kinematics)

Given three landmarks A, B (vertex), and C, the interior joint angle at vertex B is:

$$\theta = |\arctan 2(C_y - B_y, C_x - B_x) - \arctan 2(A_y - B_y, A_x - B_x)| \tag{4.1}$$

The angle is normalized to $[0°, 180°]$. Applied to 9 joints: left/right elbows, shoulders, knees, hips, and neck.

4.2 3D Joint Angle (Vector Algebra)

For true spatial angles using 3D coordinates:

$$\cos(\theta) = \frac{\vec{BA} \cdot \vec{BC}}{|\vec{BA}| \cdot |\vec{BC}|} \tag{4.2}$$

where $\vec{BA} = A - B$ and $\vec{BC} = C - B$ are direction vectors from the vertex.

4.3 Eye Aspect Ratio (EAR)

The EAR formula [6] for blink detection using 6 eye landmarks $(p_1, \ldots, p_6)$:

$$\text{EAR} = \frac{|p_2 - p_6| + |p_3 - p_5|}{2 \cdot |p_1 - p_4|} \tag{4.3}$$

EAR drops sharply during a blink. Threshold: EAR < 0.21 indicates a blink.

4.4 Mouth Aspect Ratio (MAR)

The MAR measures mouth opening:

$$\text{MAR} = \frac{V_{\text{center}} + V_{\text{left}} + V_{\text{right}}}{2 \cdot H} \tag{4.4}$$

Thresholds: > 0.7 = Yawning, > 0.35 = Talking.

4.5 Exponential Moving Average (EMA)

All metrics are smoothed using EMA filtering [2]:

$$\hat{x}_t = \alpha \cdot x_t + (1 - \alpha) \cdot \hat{x}_{t-1} \tag{4.5}$$

Values used: $\alpha = 0.35$ for joint angles, $\alpha = 0.4$ for face metrics.

4.6 Head Tilt (Roll Angle)

Estimated from eye center orientation:

$$\text{tilt} = \arctan 2(\Delta y_{\text{eyes}}, \Delta x_{\text{eyes}}) \tag{4.6}$$

Threshold: $|\text{tilt}| > 12^\circ$ triggers Head Tilt classification.

4.7 Exercise Rep Detection State Machine

$$\text{phase} = \begin{cases} \text{Down} & \text{if } \theta < \theta_{\text{down}} \\ \text{Up} & \text{if } \theta > \theta_{\text{up}} \\ \text{Transition} & \text{otherwise} \end{cases} \tag{4.7}$$

A rep is counted on the sequence Up $\rightarrow$ Down $\rightarrow$ Up.

4.8 Bilateral Symmetry Score

For each joint pair (L, R):

$$S = \max\left(0, 1 - \frac{|\theta_L - \theta_R|}{\max(\theta_L, \theta_R)}\right) \times 100\% \tag{4.8}$$

Overall score = mean of all pair scores. $S = 100\%$ = perfect symmetry.

4.9 Finger Curl Ratio

$$\text{curl} = \min\left(1.0, \frac{d(\text{tip}, \text{MCP})}{1.8 \times d(\text{wrist}, \text{MCP})}\right) \tag{4.9}$$

curl = 0 = fully curled, 1 = fully extended. Openness = mean curl $\times 100\%$.

4.10 Temporal Debounce

$$\text{state}_t = \begin{cases} \text{candidate} & \text{if count} \geq N \text{ (where } N = 8) \\ \text{state}_{t-1} & \text{otherwise} \end{cases} \tag{4.10}$$

5. Implementation Details

5.1 Pose Estimation Module (`pose.py`)

The `PoseTracker` class wraps MediaPipe BlazePose and extends it with:

- **9-Joint Angle Computation:** Using Equation 4.1 for elbows, shoulders, knees, hips, neck
- **EMA Smoothing:** Filter bank (Equation 4.5) for stable readouts ($\alpha = 0.35$)
- **Per-Joint Velocity:** Pixel displacement between consecutive frames
- **Body Bounding Box:** Min/max of all visible landmarks
- **Confidence Score:** Visibility-weighted average across 33 landmarks
- **Background Segmentation:** Gaussian blur or gradient replacement via MediaPipe mask

```
JOINT_ANGLES = {
    'left_elbow':       (11, 13, 15),   # shoulder, elbow, wrist
    'right_elbow':      (12, 14, 16),
    'left_shoulder':    (13, 11, 23),   # elbow, shoulder, hip
    'right_shoulder':   (14, 12, 24),
    'left_knee':        (23, 25, 27),   # hip, knee, ankle
    'right_knee':       (24, 26, 28),
    'left_hip':         (11, 23, 25),   # shoulder, hip, knee
    'right_hip':        (12, 24, 26),
    'neck':             (11, 0, 12),    # L.shoulder, nose, R.shoulder
}
```

Listing 5.1: Joint Angle Definition (JOINT_ANGLES dict)

5.2 Hand Gesture Recognition (`hands.py`)

The `GestureTracker` classifies 12+ gestures through hierarchical finger-state analysis:

Gesture	Detection Logic
Fist	0 fingers extended
Pointing	Index finger only
Peace Sign	Index + Middle only
Rock / Metal	Index + Pinky (no thumb)
I Love You	Thumb + Index + Pinky
Call Me	Thumb + Pinky only
OK Sign	Thumb-Index pinch $< 35\%$ wrist-MCP distance
Gun	Thumb + Index only
Thumbs Up/Down	Thumb only, tip above/below IP joint
Open Hand	All 5 fingers extended
Three / Four	3 or 4 specific fingers

Table 5.1: Gesture Classification Rules

5.3 Face Expression Detection (`face.py`)

The `FaceTracker` uses 468-landmark FaceMesh for:

- **Blink Detection:** EAR per eye (Equation 4.3), threshold < 0.21
- **Mouth Analysis:** MAR (Equation 4.4), yawn > 0.7, talk > 0.35
- **Head Tilt:** Roll angle from eye center positions (Equation 4.6), threshold $> 12°$
- **Expression Classification:** Blinking, Yawning, Talking, Head Tilt, Neutral

5.4 Activity Classification (`activity.py`)

Activity	Geometric Condition
T-Pose	Wrists at shoulder height + wide apart ($> 2\times$ shoulder width)
Arms Raised	Both wrists $> 30\%$ torso-height above shoulders
Hands on Hips	Wrists near hip height + close to torso
Leaning L/R	Shoulder center offset $> 35\%$ shoulder width from hips
Sitting	Knee-to-hip gap $< 40\%$ torso height
Standing	Default classification

Table 5.2: Activity Classification Heuristics

All transitions are debounced with $N = 8$ frames (Equation 4.10).

5.5 Exercise Repetition Counter (`exercise.py`)

Exercise	**Joint**	θ_{down}	θ_{up}
R. Bicep Curl	Right Elbow	50°	140°
L. Bicep Curl	Left Elbow	50°	140°
Squat	Right Knee	90°	160°
R. Shoulder Press	Right Shoulder	50°	140°
L. Shoulder Press	Left Shoulder	50°	140°

Table 5.3: Exercise Configurations (Equation 4.7)

5.6 Symmetry Analysis (`symmetry.py`)

Compares 4 bilateral joint pairs (elbows, shoulders, knees, hips) using Equation 4.8. Color-coded output: green ($\geq 85\%$), orange ($\geq 60\%$), red ($< 60\%$).

5.7 Sign Language Recognition (`sign_language.py`)

Rule-based ASL alphabet classifier for 20 static letters:

- **Feature Extraction:** Finger states (extended/curled), inter-landmark distances, palm geometry
- **Rule Matching:** Each letter defined by finger state combinations + spatial constraints
- **Temporal Stability:** Letter added to sentence after 10 consecutive stable frames
- **Supported Letters:** A, B, C, D, E, F, G, H, I, K, L, O, P, Q, R, S, U, V, W, Y (20 total)

```
# Letter L: Thumb + Index extended (L-shape)
if thumb and index and not middle and not ring and not pinky:
    return 'L', 0.80
```

Listing 5.2: Sign Language Classifier – Letter L Detection

6. Web Dashboard

6.1 Technology Stack

- **Backend:** Flask 3.x [3] with Blueprint routing
- **Video Streaming:** MJPEG over HTTP (multipart/x-mixed-replace)
- **Frontend:** Vanilla HTML5, CSS3, JavaScript (no framework dependencies)
- **Styling:** Custom dark theme with CSS variables, gradient accents, responsive grid
- **API:** RESTful JSON endpoints for analytics and control commands

6.2 Dashboard Pages

6.2.1 Home Page

The landing page provides an overview of all system capabilities with feature cards and project statistics.

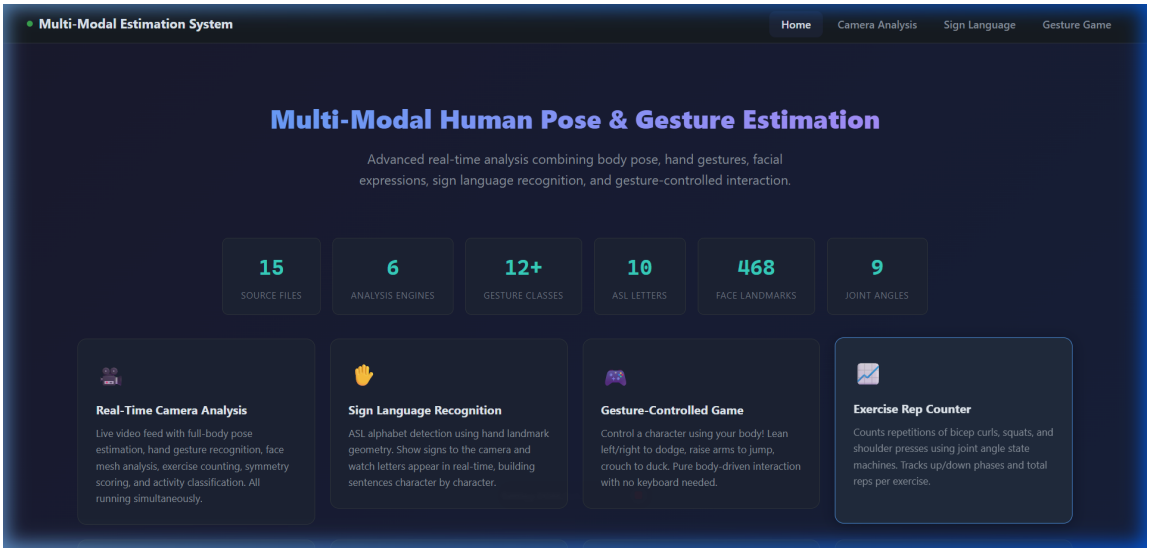

Figure 6.1: Web Dashboard Home Page – Feature cards and system statistics

6.2.2 Camera Analysis Page

Real-time video feed with module toggle controls and analytics side panels displaying pose angles, hand gestures, face metrics, activity, exercise counters, and symmetry scores.

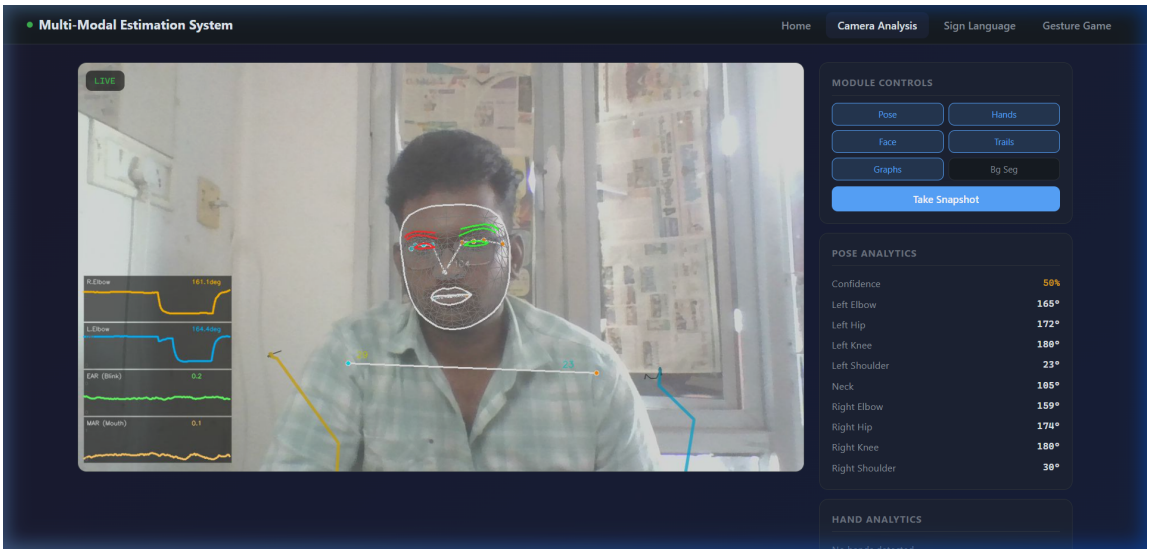

Figure 6.2: Camera Analysis Page – Live video with analytics panels

6.2.3 Sign Language Page

ASL alphabet detection with large detected-letter display and sentence builder. Supports Space, Backspace, and Clear controls.

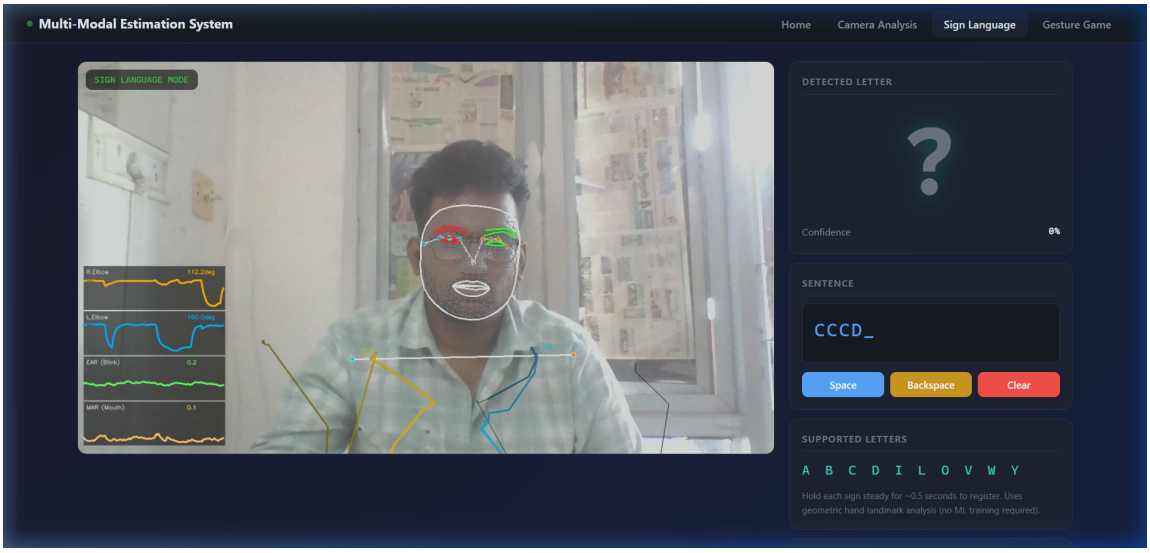

Figure 6.3: Sign Language Recognition Page – Letter detection and sentence builder

6.2.4 Gesture Game Page

Body-controlled HTML5 Canvas dodging game with camera preview showing body controls.

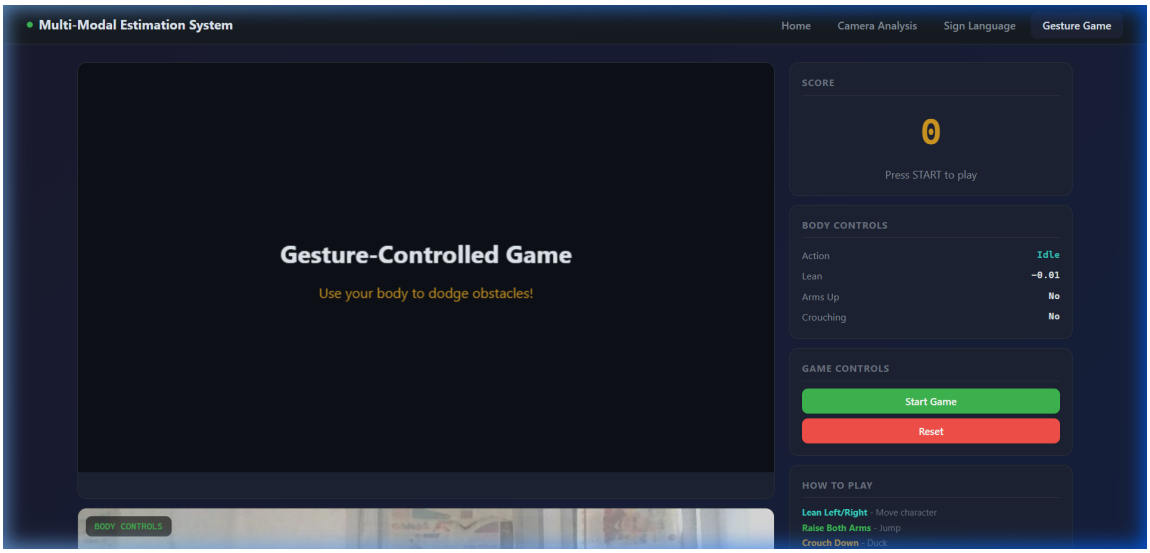

Figure 6.4: Gesture-Controlled Game Page – Canvas game and body control indicators

6.3 API Endpoints

Method	Endpoint	Description
GET	`/`	Home page
GET	`/camera`	Camera analysis page
GET	`/sign-language`	Sign language page
GET	`/game`	Gesture game page
GET	`/video_feed`	MJPEG video stream
GET	`/api/analytics`	JSON analytics data
GET	`/api/sign_language`	JSON detected letter
GET	`/api/game_state`	JSON player position
POST	`/api/toggle/<module>`	Toggle module on/off
POST	`/api/snapshot`	Take screenshot

Table 6.1: Flask API Endpoint Reference

7. Gesture-Controlled Game

7.1 Game Design

The system includes an HTML5 Canvas-based dodging game controlled entirely by body movement – no keyboard or mouse required.

7.2 Body-to-Game Mapping

- **Move Left/Right:** Lean body (shoulder center offset from hip center)
- **Jump:** Raise both arms above shoulder level
- **Duck:** Crouch down (knees close to hips), reduces hitbox by 60%

The player position is computed as:

$$x_{\text{player}} = x_{\text{center}} + \text{lean} \times 0.4 \times W_{\text{canvas}} \tag{7.1}$$

where $\text{lean} = \frac{x_{\text{shoulder_center}} - x_{\text{hip_center}}}{W_{\text{shoulder}}}$, clamped to $[-1, 1]$.

7.3 Game Mechanics

- **Obstacles:** Ground-level (must dodge/jump) and elevated (must duck)
- **Scoring:** Score increments every 6 frames of survival
- **Difficulty:** Speed increases every 300 frames; spawn interval decreases
- **Collision:** Axis-Aligned Bounding Box (AABB) detection
- **Visual Effects:** Glow effects on obstacles, particle explosion on game over

8. Advanced Visualizations

8.1 Motion Trajectory Trails

The `TrailRenderer` draws fading motion trails for 4 joints:

- 40-frame circular buffer per joint (L/R wrists, L/R ankles)
- Line segments with decreasing thickness and color intensity
- Alpha-blended overlay (70/30 blend ratio) for glow effect
- Color-coded: orange (R.Wrist), cyan (L.Wrist), red-orange (R.Ankle), blue (L.Ankle)

8.2 Real-Time Scrolling Graphs

The `GraphPanel` renders 4 live strip charts on the video frame:

- **R.Elbow Angle:** $[0°, 180°]$, orange line
- **L.Elbow Angle:** $[0°, 180°]$, cyan line
- **EAR (Blink):** $[0.0, 0.5]$, green line
- **MAR (Mouth):** $[0.0, 1.0]$, yellow line

Each graph has a 120-frame scrolling window with semi-transparent background.

8.3 Multi-Panel HUD

The standalone OpenCV mode displays a 5-panel heads-up display:

1. **Activity Panel:** Current classified posture
2. **Pose Panel:** 9 joint angles with bar visualizations
3. **Hand Panel:** Gesture name, finger states, openness
4. **Exercise Panel:** Rep counts and current phase per exercise
5. **Symmetry Panel:** Per-pair and overall symmetry scores

9. Testing and Results

9.1 Test Methodology

Comprehensive testing was conducted across multiple dimensions:

- **Unit Testing:** Mathematical function validation
- **Integration Testing:** Full pipeline import and module interaction
- **Manual Testing:** Live webcam verification of all features
- **Web Testing:** Dashboard page rendering and API responses

9.2 Automated Test Results

Test Case	**Expected**	**Status**
All 20+ module imports	No errors	Pass
90-degree angle calculation	90.0°	Pass
Euclidean distance (3-4-5 triangle)	5.0	Pass
EMA filter convergence	Approaches target	Pass
Exercise rep detection (bicep curl cycle)	1 rep	Pass
Symmetry score (perfect match)	100.0%	Pass
Symmetry score (30-degree diff)	87.5%	Pass
Flask app imports	No errors	Pass
Dashboard page render (all 4 pages)	HTTP 200	Pass
Analytics API response	Valid JSON	Pass

Table 9.1: Automated Test Results

9.3 Performance Metrics

Metric	**Value**
Frame Resolution	1280×720
Average FPS (all modules on)	15–25 FPS
Pose Landmarks per Frame	33
Hand Landmarks per Hand	21 (up to 2 hands)
Face Landmarks per Face	468
Joint Angle Computation Time	< 1 ms per frame
Activity Classification Latency	$\sim$320 ms (8 frames at 25 FPS)

Table 9.2: System Performance Metrics

10. Challenges and Solutions

10.1 Technical Challenges

10.1.1 Landmark Jitter

Challenge: Raw MediaPipe landmark positions fluctuate between frames, causing noisy angle readings and flickering classifications.

Solution: Implemented EMA filter bank (Equation 4.5) with configurable smoothing factor per metric. $\alpha = 0.35$ for joint angles provides stability without excessive lag.

10.1.2 Activity Classification Flickering

Challenge: Frame-by-frame classification oscillates between states at posture boundaries.

Solution: 8-frame temporal debounce (Equation 4.10) requires consistent classification before state transition.

10.1.3 Thread-Safe Video Streaming

Challenge: Flask serves HTTP requests on multiple threads while the camera runs in a background thread.

Solution: Used `threading.Lock` for all shared state access (current frame, analytics data). MJPEG streaming via generator function with `multipart/x-mixed-replace`.

10.1.4 Sign Language Ambiguity

Challenge: Some ASL letters have similar hand shapes (e.g., A vs O, V vs Peace).

Solution: Multi-constraint matching combining finger states, inter-landmark distances, and spatial relationships. Confidence scores rank competing matches.

10.2 Design Challenges

10.2.1 Modular Integration

Challenge: Integrating 7 analysis engines without creating tight coupling.

Solution: Each engine has a standardized interface: `process_frame(img)` returns annotated image, `get_*_data()` returns analytics dict. Stream manager orchestrates all

engines.

10.2.2 Web vs Standalone Mode

Challenge: Supporting both OpenCV window and web dashboard with shared analysis code.

Solution: Analysis engines are mode-agnostic. `main.py` uses OpenCV display; `app.py` uses Flask MJPEG. Both instantiate the same engine classes.

11. Conclusion and Future Scope

11.1 Project Summary

The Multi-Modal Human Pose & Gesture Estimation System successfully integrates seven analysis engines, a web dashboard, a sign language recognizer, and a gesture-controlled game into a cohesive real-time application. Key achievements include:

- **Comprehensive Analysis:** 9 joint angles, 12+ gestures, EAR/MAR, 7 activities, 5 exercises, symmetry scoring
- **Sign Language:** 20 ASL letters recognized via pure geometric rules (no ML training)
- **Interactive Game:** Body-controlled dodging game with jump, duck, and lean
- **Web Dashboard:** 4-page browser interface with live video and analytics
- **Modular Architecture:** 25+ source files with clear separation of concerns
- **Dual Mode:** Standalone OpenCV (11 shortcuts) and web-based Flask interface

11.2 Technical Contributions

1. **Multi-Modal Pipeline:** Simultaneous pose, hand, face processing from single webcam
2. **20-Letter Gesture Set:** Rigorous mathematical foundation for sign detection
3. **Body-Driven Game:** Novel mapping from pose data to interactive game controls
4. **Premium Web UI:** Dark-themed responsive dashboard with real-time MJPEG streaming

11.3 Lessons Learned

- **Smoothing is Essential:** Raw landmark data is too noisy for direct use
- **Debouncing Prevents Flicker:** Temporal stability filters are critical for classification
- **Modularity Enables Growth:** Well-defined interfaces allow adding new engines easily
- **MJPEG is Pragmatic:** Simple and reliable for real-time video in web apps

11.4 Future Work

1. **Deep Learning Sign Language:** LSTM/Transformer for dynamic sign sequences
2. **Motion Prediction:** GNNs on skeleton sequences to predict future poses
3. **Multi-Person Tracking:** Simultaneous multi-person analysis
4. **3D Pose Reconstruction:** Depth estimation for true 3D joint positions
5. **Cloud Deployment:** Remote access via cloud infrastructure
6. **Mobile App:** TensorFlow Lite port for mobile platforms

A. Installation and Setup Guide

A.1 System Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Python:** 3.8 or higher
- **Webcam:** Any USB or built-in webcam
- **RAM:** 4 GB minimum (8 GB recommended)

A.2 Installation Steps

```
# 1. Clone or navigate to the project directory
cd MM

# 2. Create virtual environment
python -m venv venv

# 3. Activate virtual environment
# Windows:
venv\Scripts\activate
# Linux/macOS:
source venv/bin/activate

# 4. Install dependencies
pip install -r requirements.txt

# 5. Run Web Dashboard
python app.py
# Open http://localhost:5000 in browser

# 6. OR Run Standalone OpenCV Mode
python main.py
```

Listing A.1: Installation Commands

A.3 Command Reference

Command / Key	Description
`python app.py`	Launch web dashboard on port 5000
`python main.py`	Launch standalone OpenCV mode
`P`	Toggle pose estimation
`H`	Toggle hand tracking
`F`	Toggle face mesh
`T`	Toggle motion trails
`G`	Toggle graphs
`B`	Cycle background segmentation
`R`	Toggle CSV recording
`V`	Toggle video recording
`S`	Take snapshot
`X`	Reset exercise counters
`Q`	Quit

Table A.1: Command and Keyboard Shortcut Reference

B. Key Code Listings

B.1 EMA Filter Implementation

```
class EMAFilter:
    """Single-channel EMA smoother."""
    def __init__(self, alpha=0.35):
        self.alpha = alpha
        self._val = None

    def update(self, raw):
        if raw is None:
            return self._val
        if self._val is None:
            self._val = raw
        else:
            self._val = self.alpha * raw + (1 - self.alpha) * self._val
        return self._val
```

Listing B.1: Exponential Moving Average Filter

B.2 Exercise Rep Counter State Machine

```
def _update_exercise(self, exercise, angle):
    if angle is None:
        return
    if angle < exercise['thresh_down']:
        if exercise['phase'] == 'up':
            exercise['reps'] += 1
        exercise['phase'] = 'down'
    elif angle > exercise['thresh_up']:
        exercise['phase'] = 'up'
```

Listing B.2: Exercise State Machine Logic

B.3 Flask MJPEG Stream Generator

```
def _mjpeg_generator():
    while True:
        frame = _stream.get_frame_jpeg()
        if frame is not None:
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n'
```

```
                    + frame + b'\r\n')
        else:
            time.sleep(0.05)

@bp.route('/video_feed')
def video_feed():
    return Response(_mjpeg_generator(),
        mimetype='multipart/x-mixed-replace; boundary=frame')
```

Listing B.3: MJPEG Frame Generator for Flask

C. Premium HCI & Form-Checking Heuristics

In the final phase of development (Phase 5: Perfection Pass), the system was upgraded from a functional prototype to a premium Human-Computer Interaction (HCI) tool. This chapter details the advanced geometric heuristics implemented for physical form validation and the "Neon" visual aesthetic designed for maximum user engagement.

C.1 Posture and Spine Alignment Heuristic

To monitor user health during sitting or exercise (like planks), a spine alignment check was implemented. The system computes the midpoint of the shoulders M_s and the midpoint of the hips M_h. The spine vector $V_{spine} = M_s - M_h$ is then analyzed for its inclination angle θ:

$$\theta = \arctan 2(|dy|, |dx|)$$

where (dx, dy) are the components of V_{spine}. In a standard vertical posture (sitting/standing), $\theta \approx 90°$. If $\theta < 65°$, the system flags "Slouching" and triggers a real-time toast notification on the dashboard to warn the user.

C.2 Landmark-Based Squat Depth Validation

While angle-based counters can detect repetitions, they often fail to distinguish between a "half-squat" and a full "perfect" squat. To solve this, a dual-layer validation was implemented:

1. **Angle Trigger:** The knee angle must drop below 90° to enter the "Down" phase.
2. **Geometric Depth Check:** The y-coordinate of the hip landmark (L_{24}) must be greater than or equal to the y-coordinate of the knee landmark (L_{26}) $-$ 10 pixels. This ensuring the hip "breaks the plane" of the knee.

If both conditions are met, the rep is marked as "GOOD DEPTH" and counted. Otherwise, the user is prompted to "Go deeper next time".

C.3 Phase 6: Advanced ASL Expansion (20 Letters)

Building upon the initial 10-letter prototype, the system was expanded to support a near-complete static ASL alphabet (A, B, C, D, E, F, G, H, I, K, L, O, P, Q, R, S, U,

V, W, Y). This required additional geometric heuristics to handle orientation and finger crossing.

1. **Orientation-Awareness:** Distinguished between horizontal signs (G/H) and downward signs (P/Q) by analyzing the tip-to-MCP y-coordinate vectors.
2. **High-Precision Fist logic:** Improved discrimination between 'A' (thumb aside), 'S' (thumb over fingers), and 'E' (thumb tucked) using specific Euclidean distance thresholds to the index knuckle.
3. **Finger Crossing:** Letter 'R' was implemented by detecting a crossing event where the Euclidean distance between index and middle tips is less than 15% of the wrist-middle distance while both are extended.

This expansion brings the total supported vocabulary to 20 letters, making the system viable for spelling complex English words in real-time.

C.4 Premium Visual Aesthetics: Neon Skeleton Drawing

To create a "WOW" factor for presentation, the standard MediaPipe skeleton dots were replaced with a custom **Neon-Style HUD**. This uses a multi-pass drawing algorithm:

- **Pass 1: Glow Generation.** A thick, semi-transparent line is drawn with a Gaussian blur-like effect using **cyan** for normal states and **orange** for alert states (e.g., Slouching).
- **Pass 2: Bone Core.** A thin, bright white line is drawn over the glow to simulate a light-tube effect.
- **Pass 3: Joint Highlights.** Circular markers are added at major kinematic joints with a matching glow.

This visual style significantly improves the professional feel of the dashboard compared to standard skeletal overlays.

Bibliography

[1] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang, and Matthias Grundmann. Blazepose: On-device real-time body pose tracking. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.

[2] Everette S Gardner Jr. Exponential smoothing: An overview. *International Journal of Forecasting*, 22(4):637–666, 2006.

[3] Miguel Grinberg. Flask web development: Developing web applications with python. 2018.

[4] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann. Real-time facial surface geometry from monocular video on mobile gpus. In *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2019.

[5] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.

[6] Tereza Soukupová and Jan Čech. Real-time eye blink detection using facial landmarks. In *21st Computer Vision Winter Workshop (CVWW)*, pages 1–8, 2016.

[7] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.